

**FACULTY OF NATURAL SCIENCE**  
**DEPARTMENT OF COMPUTER SCIENCE**

**CSE2102: Database Systems Design and Information Management**  
**Semester Project: Essequibo River Crossing Management System**  
**(ERCMS)**



**Group Members:**

Kaylee Carter – 1051799  
Ravindra Persaud – 1052507  
Vimla Persaud – 1052339  
Tulsidai Singh – 1052682

**Lecturers:** Ms. Amrita Ramnauth & Mr. Phillip Gajadhar

**Date of Submission:** December 27, 2025

# Table of Contents

|                                                                |    |
|----------------------------------------------------------------|----|
| 1. Part A: System Description and Conceptual Design.....       | 3  |
| 1.1 Domain Description .....                                   | 3  |
| 1.2 Entity-Relationship Diagram (ERD).....                     | 5  |
| 2. Part B: Logical Database Design .....                       | 6  |
| 2.1 Relational Schema .....                                    | 7  |
| 2.2 Functional Dependencies .....                              | 8  |
| 2.3 Normalization Analysis.....                                | 10 |
| 2.4 Resulting Normalized Schema.....                           | 11 |
| 3. Part C: Implementation and Population.....                  | 12 |
| 3.1 Database Implementation.....                               | 12 |
| 3.2 Data Generation Strategy .....                             | 12 |
| 4. Part D: SQL Queries and Reports .....                       | 13 |
| Category 1: Basic Queries (Filters & Comparison).....          | 13 |
| 1. Track the New Vessel.....                                   | 13 |
| 2. Freight Vehicles .....                                      | 13 |
| Category 2: Joins (Inner, Left, Right) .....                   | 14 |
| 3. Passengers (Inner Join) .....                               | 14 |
| 4. Crew Roster for a Trip (Three-way Join).....                | 15 |
| Category 3: Aggregation & Group By .....                       | 16 |
| 5. Revenue Per Route.....                                      | 16 |
| 6. Vessel Use .....                                            | 17 |
| Category 4: Sorting, Ordering, and Limiting.....               | 18 |
| 7. Recent Bookings .....                                       | 18 |
| 8. Highest Paid Crew Members .....                             | 19 |
| Category 5: Complex Reporting (Subqueries & Multi-Table) ..... | 20 |
| 9. The “Ghost Vehicle” Audit (Data Integrity Check) .....      | 20 |
| 10. Find Schedules with Above-Average Revenue .....            | 21 |
| 5. Mark Distribution Form.....                                 | 22 |

# **1. Part A: System Description and Conceptual Design**

## **1.1 Domain Description**

The Parika Ferry System Database is designed to support the operational management of ferries operated by Guyana's Transport and Harbours Department (T&HD) along the Essequibo River corridor, specifically between Parika, Supenaam and Bartica. These ferries, including MV Kanawan, MV Sabanto, MV Malali, and the recently commissioned MV Konawaruk 1899, transport both passengers and vehicles on scheduled sailings each day. Given the high volume of travelers and vehicles, particularly during peak travel periods, an organized and reliable database system is required to efficiently manage ferry operations and related data. The main purpose of the database is to store information on vessels, terminals, routes, schedules, people, bookings, tickets, vehicles, and crews. Every vessel operates within its passenger and vehicle capacity, and this capacity is maintained in the database through specific rules during booking and ticket issuance. The vehicles that are carried by the ferry are classified, for example, cars and trucks, and different ticket amounts are charged based on their classification. The tickets issued for different passengers, crews selected for specific vessels at a particular time and route, as well as vehicles that have been transported, are all tracked within this system.

Central to this model is the Person entity, which generalizes all people interacting with the ferry system. Every person participates either as a Passenger or as a Crew member; this is modeled using an Is-A relationship-an inheritance relationship. Common personal information is maintained at the Person level, while role-specific attributes are maintained at the subtypes. Passengers maintain travel-related identification attributes, while crew members maintain employment-related information such as rank and salary. Crew members are assigned to scheduled sailings via the crew assignment relationship; each scheduled sailing may have multiple crew assignments and vice versa, thus enabling multiple crew members on one trip and role-based staffing.

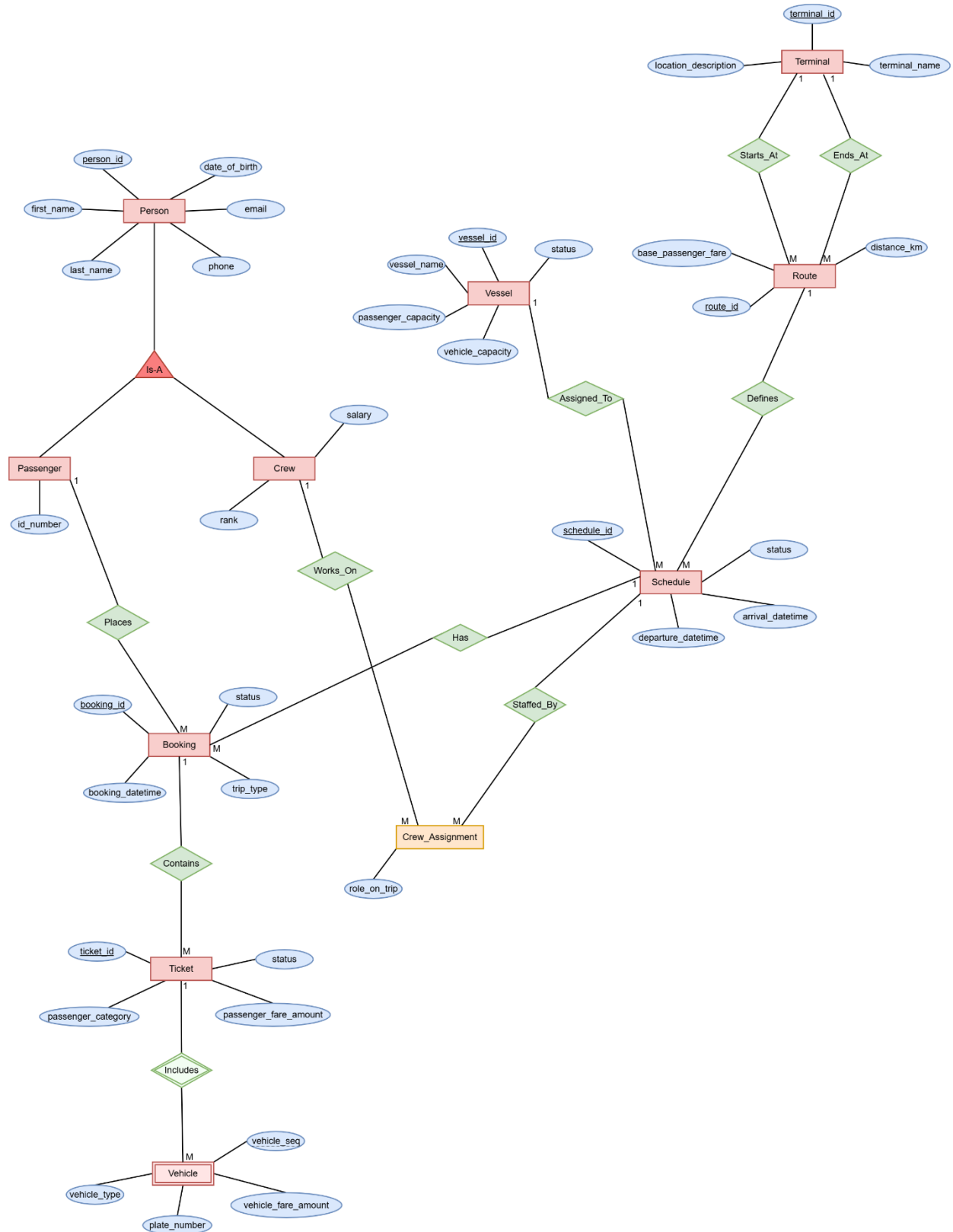
Routes are the directional ferry paths between terminals, like Parika to Supenaam or Supenaam to Parika. Direction is modelled here as an important aspect of the domain because sailings in the opposite direction constitute different routes, possibly differing in scheduling and operational analysis. Consequently, every route has precisely one origin terminal and one destination terminal, and every route has multiple scheduled sailings. A Schedule signifies a specific instance of a sailing, thus connecting a vessel to a route at a specified departure and arrival time.

Passenger travel is organized through the Booking entity, which represents the transaction made by a passenger for a specific scheduled sailing. One or many bookings may be made by a passenger, and each will be tied to exactly one schedule. Tickets will be issued on the basis of the booking, enabling many tickets to be generated in the context of a single transaction. This will enable group travel and maintain a strict separation of the concepts of schedule, transaction, and the record of the passenger or the vehicles.

The database also includes a Vehicle entity, which is modeled as a weak entity because a vehicle cannot exist in the system unless it is associated with a valid ticket. Vehicles are identified using a partial key combined with the ticket identifier, reflecting the real-world constraint that vehicles are only recorded when booked for transport. If a ticket is cancelled or removed, all vehicles linked to that ticket are automatically removed, preserving data integrity.

Overall, the Parika Ferry System Database is substantial enough to include multiple entity sets and relationship types, including one-to-many, many-to-many, and specialization relationships, while remaining manageable and realistic. Unique aspects of the domain include directional route modeling, the inclusion of both passengers and vehicles with capacity and pricing constraints, structured crew assignment per sailing, and the natural use of weak entities and inheritance. These characteristics make the database both academically rigorous and well suited for real-world ferry operations.

## 1.2 Entity-Relationship Diagram (ERD)



## **2. Part B: Logical Database Design**

In this aspect of the development process, the logical database design is derived from the Entity Relationship Diagram (ERD) done in Part A. Each entity and relationship presented in the conceptual model is then transformed into a set of relational tables. Primary keys are applied to uniquely identify tuples, and foreign keys are used to enforce referential integrity between related relations.

The resulting relational schema was analyzed using functional dependencies to ensure that each relation is in an appropriate normal form. Where necessary, normalization principles were applied to eliminate redundancy and prevent update, insertion, and deletion anomalies.

## 2.1 Relational Schema

The following relations were derived directly from the Entity Relationship Diagram (ERD):

- **vessel**  
(vessel\_id PK, vessel\_name UNIQUE, passenger\_capacity, vehicle\_capacity, status)
- **terminal**  
(terminal\_id PK, terminal\_name UNIQUE, location\_description)
- **route**  
(route\_id PK, origin\_terminal\_id FK, destination\_terminal\_id FK, distance\_km, base\_passenger\_fare)
- **schedule**  
(schedule\_id PK, route\_id FK, vessel\_id FK, departure\_datetime, arrival\_datetime, status)
- **booking**  
(booking\_id PK, person\_id FK, schedule\_id FK, booking\_datetime, trip\_type, status)
- **person**  
(person\_id PK, first\_name, last\_name, phone, email, date\_of\_birth)
- **passenger**  
(person\_id PK/FK, id\_number UNIQUE)
- **crew**  
(person\_id PK/FK, crew\_rank, salary)
- **ticket**  
(ticket\_id PK, booking\_id FK, passenger\_category, passenger\_fare\_amount, status)
- **vehicle** (weak entity)  
(ticket\_id PK/FK, vehicle\_seq PK, vehicle\_type, plate\_number, vehicle\_fare\_amount)
- **crew\_assignment**  
(schedule\_id PK/FK, crew\_person\_id PK/FK, role\_on\_trip)

The passenger and crew relations used an Is-A (inheritance) hierarchy from the person supertype. The vehicle relation is modeled as a weak entity and is identified by the combination of the owner key ticket\_id and the partial key vehicle\_seq.

## 2.2 Functional Dependencies

The functional dependencies listed below describe how attributes depend on each other in the Essequibo River Crossing Management System (ERCMS) database and are used to ensure the database is well-designed and normalized.

- **vessel**  
vessel\_id -> vessel\_name, passenger\_capacity, vehicle\_capacity, status  
Each vessel is uniquely identified by vessel\_id, which then determines all other vessel attributes.
- **terminal**  
terminal\_id -> terminal\_name, location\_description  
Each terminal is uniquely identified by terminal\_id.
- **route**  
route\_id -> origin\_terminal\_id, destination\_terminal\_id, distance\_km, base\_passenger\_fare  
Each route is defined by its origin and destination terminals and is uniquely identified by its route\_id.
- **schedule**  
schedule\_id -> route\_id, vessel\_id, departure\_datetime, arrival\_datetime, status  
Each schedule represents a single sailing instance and is uniquely identified by schedule\_id.
- **booking**  
booking\_id -> person\_id, schedule\_id, booking\_datetime, trip\_type, status  
Each booking is uniquely identified by booking\_id, which establishes the passenger, the selected schedule, and the transaction details.
- **person**  
person\_id -> first\_name, last\_name, phone, email, date\_of\_birth  
Each person is uniquely identified by person\_id.
- **passenger**  
person\_id -> id\_number  
Each passenger is uniquely identified by person\_id, which determines the passenger identification number.
- **crew**  
person\_id -> crew\_rank, salary  
A person\_id uniquely identifies each crew member and establishes their rank and salary characteristics.

- **ticket**

ticket\_id -> booking\_id, passenger\_category, passenger\_fare\_amount, status

Every ticket has a unique ticket\_id and represents a single passenger slot. It references a booking\_id to link to the schedule, avoiding transitive dependencies.

- **vehicle** (weak entity)

(ticket\_id, vehicle\_seq) -> vehicle\_type, plate\_number, vehicle\_fare\_amount

A vehicle is uniquely identified by the combination of its owning ticket and its sequence number.

- **crew\_assignment**

(schedule\_id, crew\_person\_id) -> role\_on\_trip

The combination of the appointed crew member and the schedule uniquely identifies each crew assignment role.

## 2.3 Normalization Analysis

Following the functional dependencies mentioned in section B.2, each relation in the ERCMS database was examined to see if it met the First Normal Form (1NF), Second Normal Form (2NF), and Third Normal Form (3NF) requirements.

### First Normal Form (1NF)

All relations are in First Normal Form since:

- Each table has a clearly defined primary key.
- All attributes contain atomic (indivisible) values.
- There are no repeating groups or multivalued attributes within any relation.

*Example:* Single values are used to hold passenger identification numbers. Instead of grouped attributes, vehicle attributes are stored as individual rows. Therefore, all relations satisfy 1NF.

### Second Normal Form (2NF)

All relations are in Second Normal Form since:

- There are no partial dependencies in any relation with single-attribute primary keys.
- All non-key properties in any relation with composite primary keys are entirely dependent on the entire composite key.

*Example:* Attributes like vehicle\_type and plate\_number in the vehicle relation rely on the entire composite key (ticket\_id, vehicle\_seq) and not on part of it. The combination of (schedule\_id, crew\_person\_id) determines the role\_on\_trip in the crew\_assignment relation. No partial dependencies were identified.

### Third Normal Form (3NF)

All relations are in Third Normal Form since:

- There are no transitive dependencies.
- All non-key attributes depend only on the primary key.

*Example:* We identified a potential transitive dependency where Ticket could imply a Schedule. To resolve this and ensure 3NF, we removed schedule\_id from the ticket relation. The schedule is now determined solely through the booking relation (ticket -> booking -> schedule), eliminating data redundancy.

## 2.4 Resulting Normalized Schema

All the relations in the Essequibo River Crossing Management System (ERCMS) Database were confirmed to satisfy Third Normal Form (3NF) after identifying the functional dependencies and working through the normalization process.

The normalized relations used for the physical database implementation in Part C are:

- vessel
- terminal
- route
- schedule
- booking
- person
- passenger
- crew
- ticket
- vehicle
- crew\_assignment

This collection of relations serves as a consistent basis for the physical database implementation and reflects the completely normalized logical database architecture.

## **3. Part C: Implementation and Population**

### **3.1 Database Implementation**

The system was implemented using MySQL 8.0. We utilized InnoDB engines to enforce Foreign Key constraints and ON DELETE CASCADE rules to handle the weak entity deletions automatically.

### **3.2 Data Generation Strategy**

To simulate realistic T&HD operations, we developed a Python script (`data_generator.py`) to populate the database.

- Volume: The script generated over 1,000 distinct passenger identities.
- Logic: It programmatically creates Bookings linked to random Schedules, and then generates Tickets for those bookings.
- Integrity: The script ensures that valid `schedule_ids` are assigned to Bookings, and that Tickets are strictly child records of those Bookings, maintaining referential integrity.

## 4. Part D: SQL Queries and Reports

Below are 10 SQL queries for the Parika Ferry System.

### Category 1: Basic Queries (Filters & Comparison)

#### 1. Track the New Vessel

**Description:** We need to verify that the new vessel, *MV Konawaruk 1899*, has been added and check the status of all active boats.

```
SELECT vessel_id, vessel_name, passenger_capacity, status
FROM vessel
WHERE vessel_name LIKE '%Konawaruk%'
OR status = 'active';
```

| vessel (4r × 4c) |             |  |                   |  |                      |          |
|------------------|-------------|--|-------------------|--|----------------------|----------|
| #                | 1 vessel_id |  | 2 vessel_name     |  | 3 passenger_capacity | 4 status |
| 1                | 1           |  | MV Kanawan        |  | 400                  | active   |
| 2                | 2           |  | MV Sabanto        |  | 400                  | active   |
| 3                | 3           |  | MV Malali         |  | 250                  | active   |
| 4                | 4           |  | MV Konawaruk 1899 |  | 350                  | active   |

#### 2. Freight Vehicles

**Description:** This query helps the loading staff identify heavy vehicles (like Trucks and Canters) paying more than \$5,000, so they can prioritize them for deck space.

```
SELECT ticket_id, vehicle_type, plate_number, vehicle_fare_amount
FROM vehicle
WHERE vehicle_fare_amount > 5000
AND vehicle_type IN ('Truck', 'Canter');
```

| vehicle (1r × 4c) |             |                |                |                       |
|-------------------|-------------|----------------|----------------|-----------------------|
| #                 | 1 ticket_id | 2 vehicle_type | 3 plate_number | 4 vehicle_fare_amount |
| 1                 | 7           | Truck          | PTT-7788       | 12,000.0              |

## Category 2: Joins (Inner, Left, Right)

### 3. Passengers (Inner Join)

**Description:** This query generates a list of all passengers booked on a specific schedule (Schedule ID 1). It joins four tables to get the passenger's full name and ID number, which is required for safety regulations.

```
SELECT
  p.first_name,
  p.last_name,
  pass.id_number,
  t.status
FROM ticket t
JOIN booking b ON t.booking_id = b.booking_id
JOIN passenger pass ON b.person_id = pass.person_id
JOIN person p ON pass.person_id = p.person_id
WHERE b.schedule_id = 1;
```

| person (139r × 4c) |              |             |              |           |
|--------------------|--------------|-------------|--------------|-----------|
| #                  | 1 first_name | 2 last_name | 3 id_number  | 4 status  |
| 1                  | Alicia       | Williams    | GY-PASS-001  | cancelled |
| 2                  | Brian        | Singh       | GY-PASS-002  | active    |
| 3                  | Vimla        | Persaud     | GY-PASS-003  | cancelled |
| 4                  | Dwayne       | Hinds       | GY-PASS-010  | active    |
| 5                  | Ravindra     | Persaud     | GY-PASS-013  | active    |
| 6                  | Jennifer     | Wright      | GY-PASS-1049 | active    |
| 7                  | Jennifer     | Wright      | GY-PASS-1049 | active    |
| 8                  | Jennifer     | Wright      | GY-PASS-1049 | active    |
| 9                  | Jennifer     | Wright      | GY-PASS-1049 | active    |
| 10                 | Jennifer     | Wright      | GY-PASS-1049 | active    |
| 11                 | Donna        | Martinez    | GY-PASS-1059 | active    |
| 12                 | Donna        | Martinez    | GY-PASS-1059 | active    |
| 13                 | Donna        | Martinez    | GY-PASS-1059 | active    |
| 14                 | Donna        | Martinez    | GY-PASS-1059 | active    |
| 15                 | Lisa         | Carter      | GY-PASS-1108 | active    |
| 16                 | Ronald       | Jackson     | GY-PASS-1139 | active    |
| 17                 | Mary         | Taylor      | GY-PASS-1140 | active    |
| 18                 | Mary         | Taylor      | GY-PASS-1140 | active    |
| 19                 | Mary         | Taylor      | GY-PASS-1140 | active    |
| 20                 | Kimberly     | Scott       | GY-PASS-1146 | active    |

#### 4. Crew Roster for a Trip (Three-way Join)

**Description:** This report shows exactly which crew members are working on a specific trip (Schedule ID 1) and what their role is (e.g. Captain, Engineer) for that journey.

```
SELECT
  p.first_name,
  p.last_name,
  c.crew_rank,
  ca.role_on_trip
FROM crew_assignment ca
JOIN crew c ON ca.crew_person_id = c.person_id
JOIN person p ON c.person_id = p.person_id
WHERE ca.schedule_id = 1;
```

| person (3r × 4c) |              |             |               |                |
|------------------|--------------|-------------|---------------|----------------|
| #                | 1 first_name | 2 last_name | 3 crew_rank   | 4 role_on_trip |
| 1                | Jason        | Bowman      | captain       | captain        |
| 2                | Anika        | Sutherland  | first_officer | navigator      |
| 3                | Tiana        | Douglas     | engineer      | engineer       |

## Category 3: Aggregation & Group By

### 5. Revenue Per Route

**Description:** This report calculates the total passenger revenue generated by each route. We use a **LEFT JOIN** on the vehicle table to ensure we include tickets even if they don't have a vehicle attached.

```
SELECT
  r.route_id,
  t.terminal_name as 'Origin',
  COUNT(tk.ticket_id) as 'Tickets Sold',
  COALESCE(SUM(tk.passenger_fare_amount), 0) as 'Passenger Revenue',
  COALESCE(SUM(v.vehicle_fare_amount), 0) as 'Cargo Revenue',
  (COALESCE(SUM(tk.passenger_fare_amount), 0) +
   COALESCE(SUM(v.vehicle_fare_amount), 0)) as 'Total Revenue'
FROM ticket tk
JOIN booking b ON tk.booking_id = b.booking_id
JOIN schedule s ON b.schedule_id = s.schedule_id
JOIN route r ON s.route_id = r.route_id
JOIN terminal t ON r.origin_terminal_id = t.terminal_id
LEFT JOIN vehicle v ON tk.ticket_id = v.ticket_id
GROUP BY r.route_id, t.terminal_name;
```

| route (2r × 6c) |            |                 |                |                     |                 |                 |
|-----------------|------------|-----------------|----------------|---------------------|-----------------|-----------------|
| #               | 1 route_id | 2 Origin        | 3 Tickets Sold | 4 Passenger Revenue | 5 Cargo Revenue | 6 Total Revenue |
| 1               | 1          | Parika Stelling | 627            | 122,920.0           | 27,000.0        | 149,920.0       |
| 2               | 2          | Leguan Stelling | 388            | 69,800.0            | 12,000.0        | 81,800.0        |

## 6. Vessel Use

**Description:** This counts how many trips each vessel has successfully completed. This helps management see which boats are being used the most.

```
SELECT
  v.vessel_name,
  COUNT(s.schedule_id) as 'Trips Completed'
FROM schedule s
JOIN vessel v ON s.vessel_id = v.vessel_id
WHERE s.status = 'completed'
GROUP BY v.vessel_name;
```

| vessel (3r × 2c) |                   |                   |
|------------------|-------------------|-------------------|
| #                | 1 vessel_name     | 2 Trips Completed |
| 1                | MV Konawaruk 1899 | 1                 |
| 2                | MV Sabanto        | 1                 |
| 3                | MV Malali         | 1                 |

## Category 4: Sorting, Ordering, and Limiting

### 7. Recent Bookings

**Description:** This query shows the 10 most recent bookings entered into the system, useful for the ticket office to track real-time activity.

```
SELECT
  b.booking_id,
  p.last_name,
  b.booking_datetime,
  b.status
FROM booking b
JOIN person p ON b.person_id = p.person_id
ORDER BY b.booking_datetime DESC
LIMIT 10;
```

| booking (10r × 4c) |              |             |                     |           |
|--------------------|--------------|-------------|---------------------|-----------|
| #                  | 1 booking_id | 2 last_name | 3 booking_datetime  | 4 status  |
| 1                  | 1,009        | Singh       | 2025-12-30 00:00:00 | confirmed |
| 2                  | 1,011        | Harris      | 2025-12-30 00:00:00 | confirmed |
| 3                  | 1,028        | Hall        | 2025-12-30 00:00:00 | confirmed |
| 4                  | 1,030        | Wilson      | 2025-12-30 00:00:00 | confirmed |
| 5                  | 1,033        | Martinez    | 2025-12-30 00:00:00 | confirmed |
| 6                  | 1,064        | Allen       | 2025-12-30 00:00:00 | confirmed |
| 7                  | 1,086        | Martin      | 2025-12-30 00:00:00 | confirmed |
| 8                  | 1,090        | Gonzalez    | 2025-12-30 00:00:00 | confirmed |
| 9                  | 1,124        | Adams       | 2025-12-30 00:00:00 | confirmed |
| 10                 | 1,134        | Williams    | 2025-12-30 00:00:00 | confirmed |

## 8. Highest Paid Crew Members

**Description:** A simple HR report to find the top 5 highest-earning crew members.

```
SELECT
  p.first_name,
  p.last_name,
  c.crew_rank,
  c.salary
FROM crew c
JOIN person p ON c.person_id = p.person_id
ORDER BY c.salary DESC
LIMIT 5;
```

| person (5r × 4c) |              |             |               |           |
|------------------|--------------|-------------|---------------|-----------|
| #                | 1 first_name | 2 last_name | 3 crew_rank   | 4 salary  |
| 1                | Jason        | Bowman      | captain       | 350,000.0 |
| 2                | Anika        | Sutherland  | first_officer | 280,000.0 |
| 3                | Tiana        | Douglas     | engineer      | 260,000.0 |
| 4                | Kaylee       | Carter      | deckhand      | 180,000.0 |
| 5                | Tulsidai     | Singh       | deckhand      | 180,000.0 |

## Category 5: Complex Reporting (Subqueries & Multi-Table)

### 9. The “Ghost Vehicle” Audit (Data Integrity Check)

**Description:** This audit query is designed to detect inconsistencies by identifying vehicles that remain assigned to deck slots despite their associated tickets being cancelled. While the operational goal is zero results, the presence of records here demonstrates the system's effectiveness in flagging invalid reservations that require administrative cleanup to free up capacity.

```
SELECT
  v.plate_number,
  v.vehicle_type,
  t.status as ticket_status
FROM vehicle v
JOIN ticket t ON v.ticket_id = t.ticket_id
WHERE t.status = 'cancelled';
```

| vehicle (2r x 3c) |                |                |                 |
|-------------------|----------------|----------------|-----------------|
| #                 | 1 plate_number | 2 vehicle_type | 3 ticket_status |
| 1                 | PTT-7788       | Truck          | cancelled       |
| 2                 | PAJ-8899       | Car            | cancelled       |

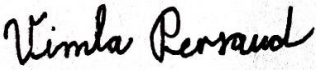
## 10. Find Schedules with Above-Average Revenue

**Description:** This uses a subquery to calculate the average revenue across all trips, and then lists the specific schedules that earned more than that average (identifying peak trips).

```
SELECT
  b.schedule_id,
  COALESCE(SUM(t.passenger_fare_amount), 0) as trip_revenue
FROM ticket t
JOIN booking b ON t.booking_id = b.booking_id
GROUP BY b.schedule_id
HAVING trip_revenue > (
  SELECT AVG(total_per_trip)
  FROM (
    SELECT COALESCE(SUM(t2.passenger_fare_amount), 0) as total_per_trip
    FROM ticket t2
    JOIN booking b2 ON t2.booking_id = b2.booking_id
    GROUP BY b2.schedule_id
  ) as subquery
);
```

| booking (5r × 2c) |               |                |  |
|-------------------|---------------|----------------|--|
| #                 | 1 schedule_id | 2 trip_revenue |  |
| 1                 | 1             | 30,020.0       |  |
| 2                 | 3             | 24,580.0       |  |
| 3                 | 4             | 24,400.0       |  |
| 4                 | 5             | 26,300.0       |  |
| 5                 | 6             | 27,720.0       |  |

## 5. Mark Distribution Form

| USI     | %                                                                                             | Signature                                                                           | Date       |
|---------|-----------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|------------|
| 1052339 | Part A: Conceptual Design (ER Diagram, Entities, Relationships, Constraints)<br>Video Editing |   | 27/12/2025 |
| 1052507 | Part B: Logical Design (Relational Schema, Functional Dependencies, Normalization)            |   | 27/12/2025 |
| 1051799 | Part C: Physical Design (SQL Implementation, DDL, Data Population/Python Script)              |   | 27/12/2025 |
| 1052682 | Part D: Reporting (Complex SQL Queries, Data Analysis, Report Compilation)<br>Video Editing   |  | 27/12/2025 |